Topic 6: Optionals and Optional Handling

Task 1: Library item model

```
enum BookStatus {
    case available
    case borrowed(by: String, dueIn: Int)
    case reserved(for: String)
    var label: String {
        switch self {
        case .available:
            return "Available"
        case .borrowed(let name, let days):
            return "Borrowed by \(name), due in \(days) days"
        case .reserved(let name):
            return "Reserved for \(name)"
        }
    }
}
struct Book {
    var title: String
    var status: BookStatus = .available
}
class Member {
    var name: String
    var borrowedBooks: [Book] = []

    init(name: String) {
        self.name = name
```

```
    }
}
let book1 = Book(title: "Swift Basics")
var book2 = book1
book2.status = .borrowed(by: "Ravi", dueIn: 14)


print("Original: \(book1.title) - \(book1.status.label)")
print("Copy: \(book2.title) - \(book2.status.label)")
let member1 = Member(name: "Asha")
let member2 = member1
member2.name = "Meena"
print("member1 name: \(member1.name)")
print("member2 name: \(member2.name)")
```

Output

```
Original: Swift Basics - Available
Copy: Swift Basics - Borrowed by Ravi, due in 14 days
member1 name: Meena
member2 name: Meena
```

Task 2: Shapes and areas

```
enum Shape {
    case circle(radius: Double)
    case rectangle(width: Double, height: Double)
    case triangle(base: Double, height: Double)

    var area: Double {
        switch self {
```

```
        case .circle(let radius):
            return Double.pi * radius * radius
        case .rectangle(let width, let height):
            return width * height
        case .triangle(let base, let height):
            return 0.5 * base * height
        }
    }
}
let circle = Shape.circle(radius: 5)
let rectangle = Shape.rectangle(width: 4, height: 6)
let triangle = Shape.triangle(base: 3, height: 8)
print("Circle area: \(circle.area)")
print("Rectangle area: \(rectangle.area)")
print("Triangle area: \(triangle.area)")
struct Rectangle {
    var width: Double
    var height: Double

    var perimeter: Double {
        return 2 * (width + height)
    }
    mutating func scale(by factor: Double) {
        width *= factor
        height *= factor
    }
}
let original = Rectangle(width: 4, height: 6)
var copy = original
```

```
copy.scale(by: 2)
print("Original: \(original.width) x \(original.height), perimeter \(original.perimeter)")
print("Copy: \(copy.width) x \(copy.height), perimeter \(copy.perimeter)")
```

Output

```
Circle area: 78.53981633974483
Rectangle area: 24.0
Triangle area: 12.0
Original: 4.0 x 6.0, perimeter 20.0
Copy: 8.0 x 12.0, perimeter 40.0
```

Task 3: Why does this not compile?

```
struct Counter {
    var value = 0
    mutating func increment() { value += 1 }
}
class Box {
    var value = 0
    func increment() { value += 1 }
}

var c = Counter()
c.increment()
print("Counter value: \(c.value)")
let b = Box()
b.increment()
print("Box value: \(b.value)")
let b2 = b
```

b2.increment()

print("b.value: \(b.value)")

print("b2.value: \(b2.value)")

print("Same object? \(b === b2)")

```
Counter value: 1
Box value: 1
b.value: 2
b2.value: 2
Same object? true
```